

REFLECTIVE QUESTIONS

 For this part, submit it in separate PDF files

Middleware & Architecture

1. What are the advantages of using middleware in an Express application?

Middleware helps organize backend logic into reusable and modular components. Instead of writing all logic inside route handlers, middleware separates responsibilities such as logging, validation, authentication, and error handling.

Advantages include:

- Cleaner and more readable code
- Reusability across multiple routes
- Easier debugging and maintenance
- Better scalability for larger applications
- Ability to process requests before reaching the final route handler

For example, logging middleware can automatically record every request without duplicating code in every route.

2. How does separating middleware into dedicated files improve the maintainability of your code?

Separating middleware into dedicated files improves code organization and reduces complexity. Each middleware file has a single responsibility, making the project easier to understand and maintain.

Benefits include:

- Easier to locate and update specific functionality
- Reduces duplicated code
- Encourages modular design
- Makes collaboration easier in team projects
- Simplifies testing and debugging

For example:

```
middleware/  
├── logger.js  
├── validateCredits.js  
└── auth.js
```

This structure keeps the main `server.js` file clean and focused on routing.

3. If you had to scale this API to support user roles (e.g., admin vs student), how would you modify the middleware structure?

I would create authentication and authorization middleware.

Example structure:

```
middleware/  
├── auth.js  
├── adminOnly.js  
└── studentOnly.js
```

Workflow:

1. Authentication middleware verifies the user identity using a token or session.
2. Authorization middleware checks the user role.
3. Access is granted or denied depending on permissions.

Example:

```
app.get('/admin/dashboard', auth, adminOnly, handler);
```

This structure keeps role management reusable and scalable.

Query Handling & Filtering

4. How would you handle cases where multiple query parameters conflict or are ambiguous (e.g., `minCredits=4` and `maxCredits=3`)?

The backend should validate query parameters before processing them. If conflicting values are detected, the server should return a 400 Bad Request response with a clear error message.

Example:

```
{  
  "error": "minCredits cannot be greater than maxCredits"  
}
```

This prevents invalid filtering logic and improves API reliability.

5. What would be a good strategy to make the course filtering more user-friendly (e.g., handling typos in query parameters like “falll” or “dr. smtih”)?

Several strategies can improve user-friendliness:

- Use case-insensitive matching
- Implement fuzzy string matching
- Suggest closest valid values
- Use autocomplete in frontend applications
- Validate against predefined allowed values

Example:

- "falll" → suggest "fall"
- "dr. smtih" → suggest "Dr. Smith"

Libraries such as Fuse.js can help implement fuzzy search functionality.

Security & Validation

6. What are the limitations of using a query parameter for authentication (e.g., ?token=xyz123)? What alternatives would be more secure?

Using query parameters for authentication is insecure because:

- Tokens can appear in browser history
- Tokens may be logged in server logs
- Tokens can leak through shared URLs
- Query strings are visible in many monitoring systems

More secure alternatives include:

- Authorization headers with JWT tokens
- Session-based authentication
- OAuth 2.0
- HTTP-only secure cookies

Example:

Authorization: Bearer <token>

This is safer because tokens are not exposed in URLs.

7. Why is it important to validate and sanitize query inputs before using them in your backend logic?

Validation and sanitization protect the application from invalid or malicious input.

Importance:

- Prevents crashes and unexpected behavior
- Protects against injection attacks
- Ensures data integrity
- Improves security and reliability
- Prevents logical errors

For example, without validation:

- minCredits=abc
- SQL injection attempts
- Malicious scripts

Sanitization removes or escapes dangerous characters before processing user input.

Abstraction & Reusability

8. Can any of the middleware you wrote be reused in other projects? If so, how would you package and document it?

Yes. Logging middleware and validation middleware are reusable in many Express applications.

To package them:

1. Place middleware in separate modules
2. Export functions using ES modules
3. Publish as an npm package if needed

Example:

```
export default function logger(req, res, next) {  
  console.log(req.method, req.path);  
  next();  
}
```

Documentation should include:

- Purpose
- Installation instructions
- Usage examples
- Supported options
- Expected inputs and outputs

This makes the middleware easy for other developers to use.

9. How could you design your route and middleware system to support future filters (e.g., course format, time slot)?

I would design the filtering system to be modular and extensible.

Strategies:

- Use dynamic filtering logic
- Create reusable validation middleware
- Centralize filter handling
- Store allowed filters in configuration objects

Example:

```
const filters = {  
  level,  
  semester,  
  instructor,  
  format,  
  timeSlot  
};
```

This approach allows new filters to be added with minimal code changes.

Middleware can also validate future filters independently, making the system easier to scale and maintain.